# SYSTEM DESIGN

Hospital Patient Management System (HPMS)

CS487 — Object-Oriented Design and Implementations

Sumaya Sharmin • Shreya Hada Shrestha • Ritika Khatri • Sahar Tahir • Thel Theint Theint Sen

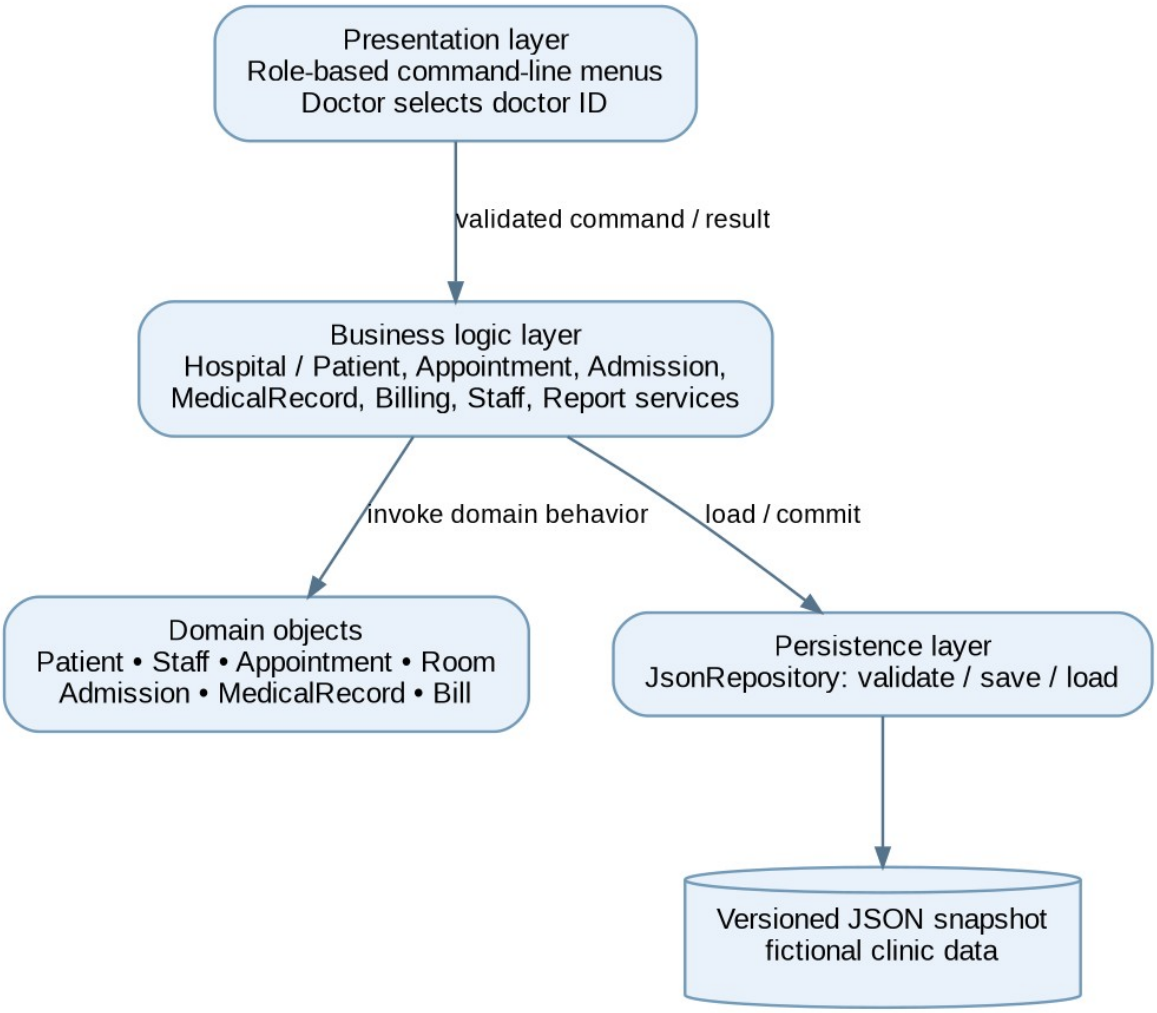
October 2026 | System design specification | Educational prototype

## 1. Purpose, scope, and design basis

This system design converts the submitted HPMS proposal, requirements collection, and user stories into testable classes, workflows, interfaces, persistence rules and UML models. The Library Management System provided in the course guidance is an example of the assignment format, not this project's subject.

- Source baseline: FR-01–FR-26; NFR-01–NFR-10; BR-01–BR-08; UC1–UC8; 28 user stories. Preserve Must Have / Should Have / Could Have priority levels.
- Core: patient and staff records, conflict-free appointments, clinical history, admission/discharge, billing and payments, reports and persistence.
- Technology and limits: Python 3, menu-driven CLI, JSON, one small clinic, fictional data, role selection without login; no SQLite, GUI, online payments, insurance or real clinical deployment.

## 2. Architecture and main components




The CLI sends commands to the service layer. Services check business invariants, work with domain objects and commit a versioned JSON snapshot through JsonRepository. Domain models are used by services; they are not an independent storage or UI layer.

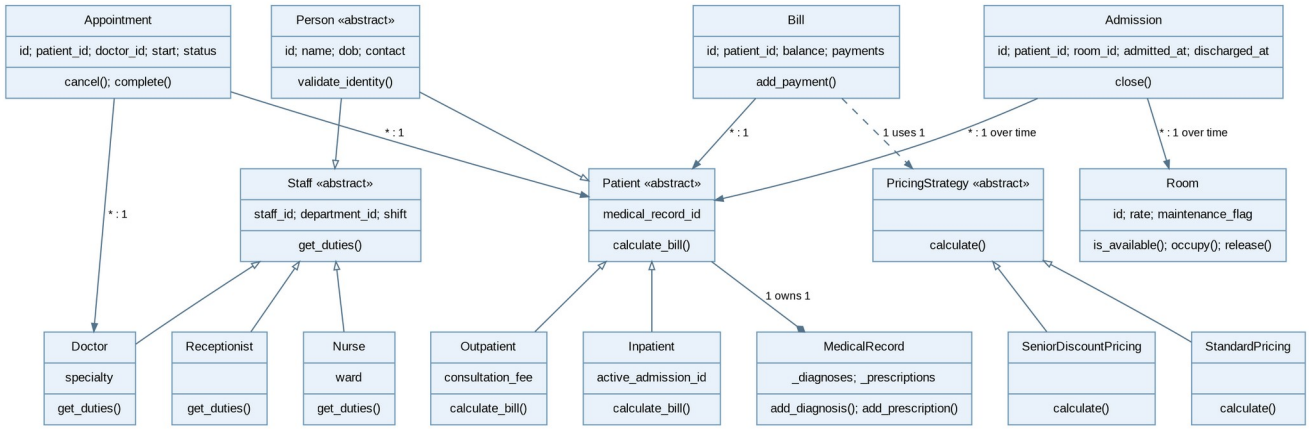
| Component | Responsibility |
|---|---|
| **Role CLI** | Role menus, doctor ID selection, typed input and output; does not modify JSON or protected data directly |
| **Hospital + services** | Patient, appointment, medical record, admission, billing, staff and reporting coordination |
| **Domain objects** | Validated state, status transitions and polymorphic behavior |
| **StaffFactory** | Creates Doctor, Nurse or Receptionist according to role |
| **PricingStrategy** | StandardPricing and SeniorDiscountPricing interchangeably calculate final charges |
| **JsonRepository** | Loads, verifies and atomically saves related records |

## 3. Domain model: classes and responsibilities

| Class / family | Core attributes | Representative methods |
|---|---|---|
| **Person (ABC)** | id, name, date_of_birth, contact | validate_identity() |
| **Patient (ABC); Outpatient, Inpatient** | medical_record_id; fee or admission context | calculate_bill() |
| **Staff (ABC); Doctor, Nurse, Receptionist** | staff_id, department_id, shift | get_duties() |
| **Appointment** | appointment_id, patient_id, doctor_id, start_time, status | cancel(), complete() |
| **Admission, Room** | admission_id, patient_id, room_id, dates; room rate, maintenance flag | close(); is_available(), occupy(), release() |
| **MedicalRecord, Prescription** | private diagnoses/prescriptions; medicine, dosage, instructions | add_diagnosis(), add_prescription() |
| **Bill; PricingStrategy (ABC)** | charges, _balance, payments; selected policy | add_payment(); calculate() |
| **Hospital; StaffFactory; JsonRepository** | domain indexes / creation / snapshots | coordinate(); create_staff(); load(), save() |

Patient lifecycle design decision: a newly registered patient is Outpatient. Admission transforms the registry's current representation into Inpatient using the same ID and MedicalRecord; discharge restores Outpatient. Separate Admission records preserve historical stays. All related objects refer to patient IDs, not cached Python object references. This is a proposed implementation policy, not a requirement previously approved by the instructor.

### 3.1 UML class diagram

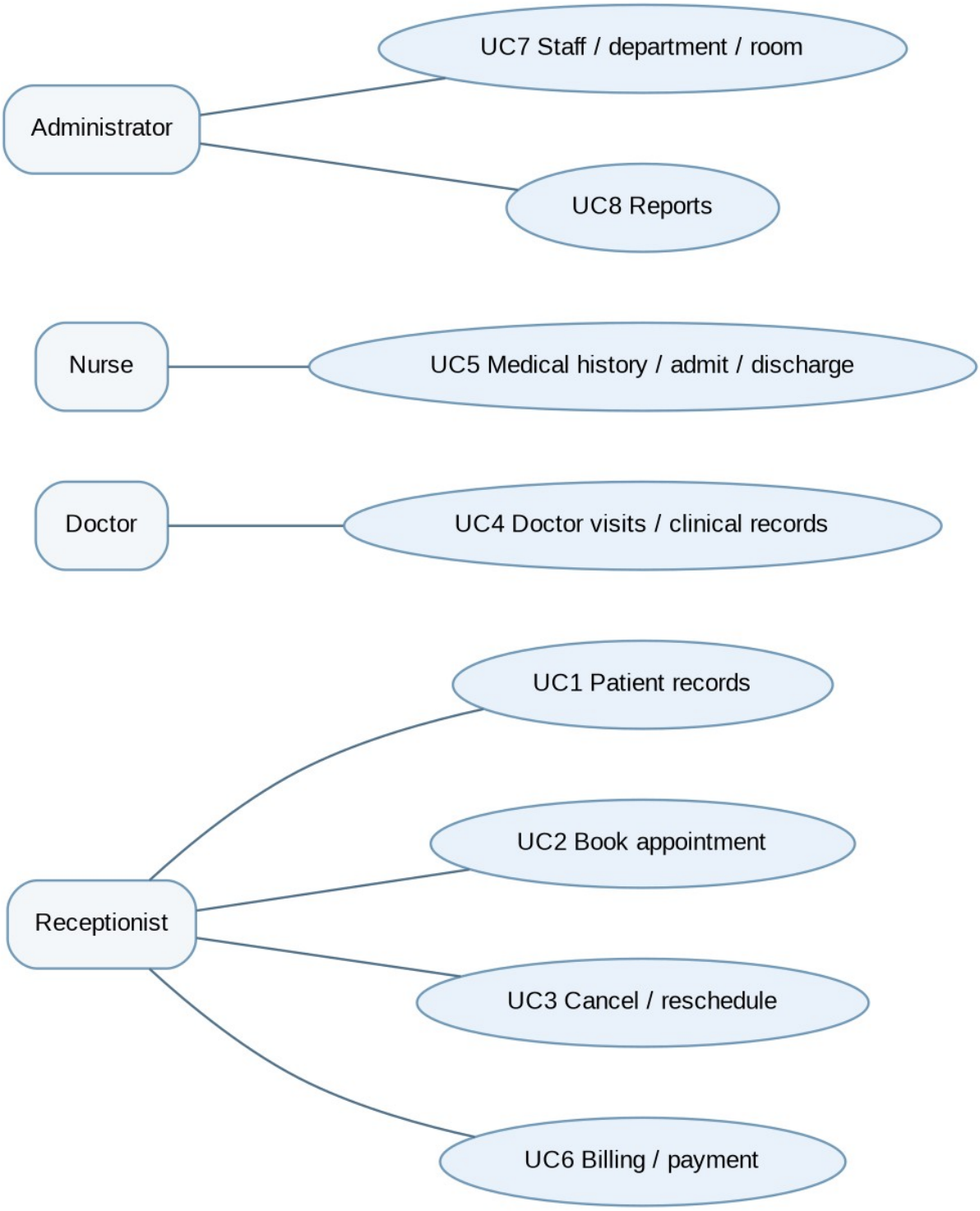



Multiplicity: each appointment links one patient and one doctor; each patient/doctor may have many appointments over time. Each admission links one patient and one room; only one active admission per patient and room. Each patient owns one medical record containing many entries. Bills use one pricing strategy.

### 3.2 OOP principles and patterns

- Encapsulation: protected diagnosis collections and bill balance change only through validated methods; role menus never edit private attributes.
- Abstraction: Python abc defines Person, Patient, Staff and PricingStrategy contracts.
- Inheritance and polymorphism: concrete Patient and Staff subclasses specialize calculate_bill() and get_duties(); concrete strategies implement calculate().
- Factory: StaffFactory centralizes staff subtype creation. Strategy: Bill delegates prices and discounts to a PricingStrategy.

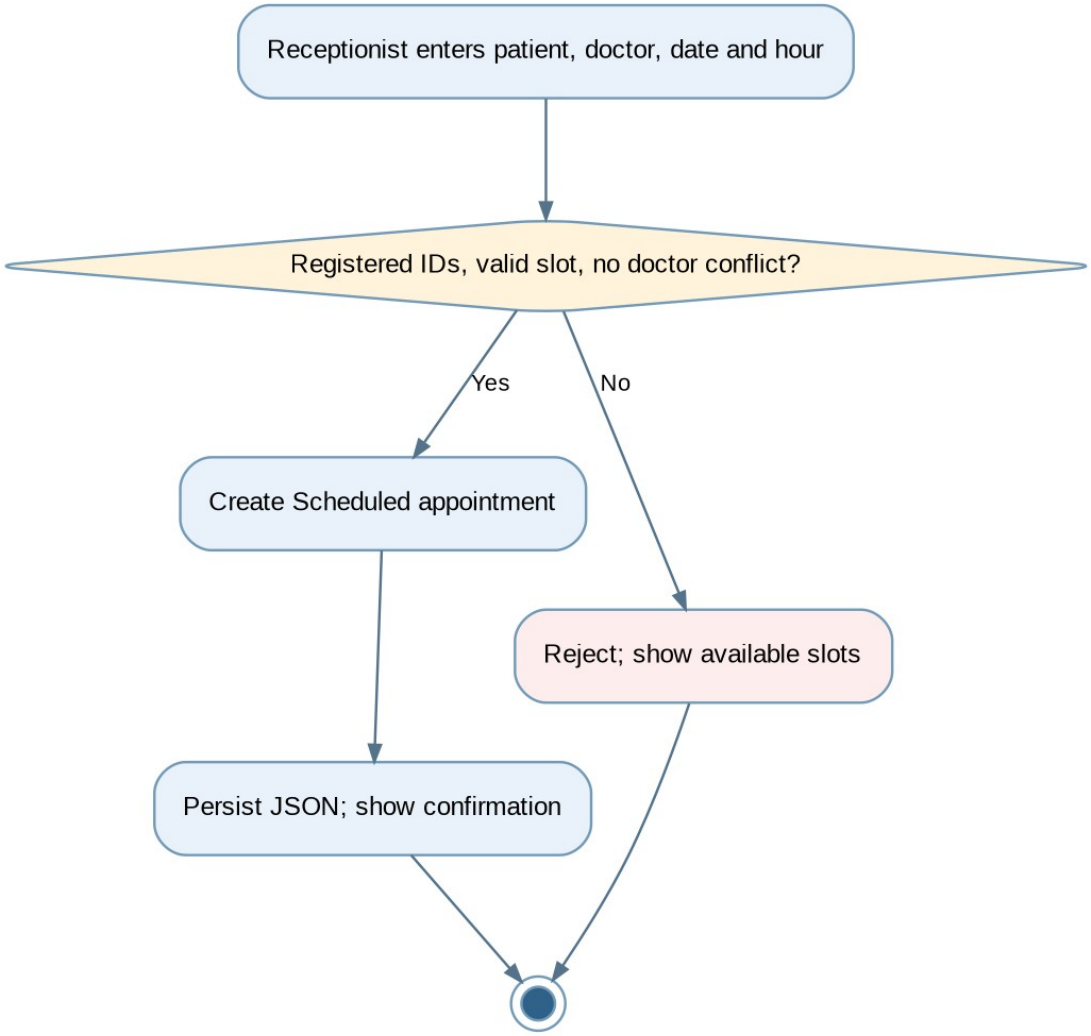
## 4. Actors, use cases, and role identification




Doctor role identification resolves instructor feedback: after selecting Doctor, the CLI prompts for a valid staff ID whose type is Doctor. All “my appointments” and new clinical entries use that selected doctor ID. No password is requested; this is simulated identity selection, not secure authentication. Nurses can only view, not edit, clinical histories in their menu.

## 5. Workflows and business rules

### 5.1 Booking, cancelling and rescheduling (UC2, UC3)

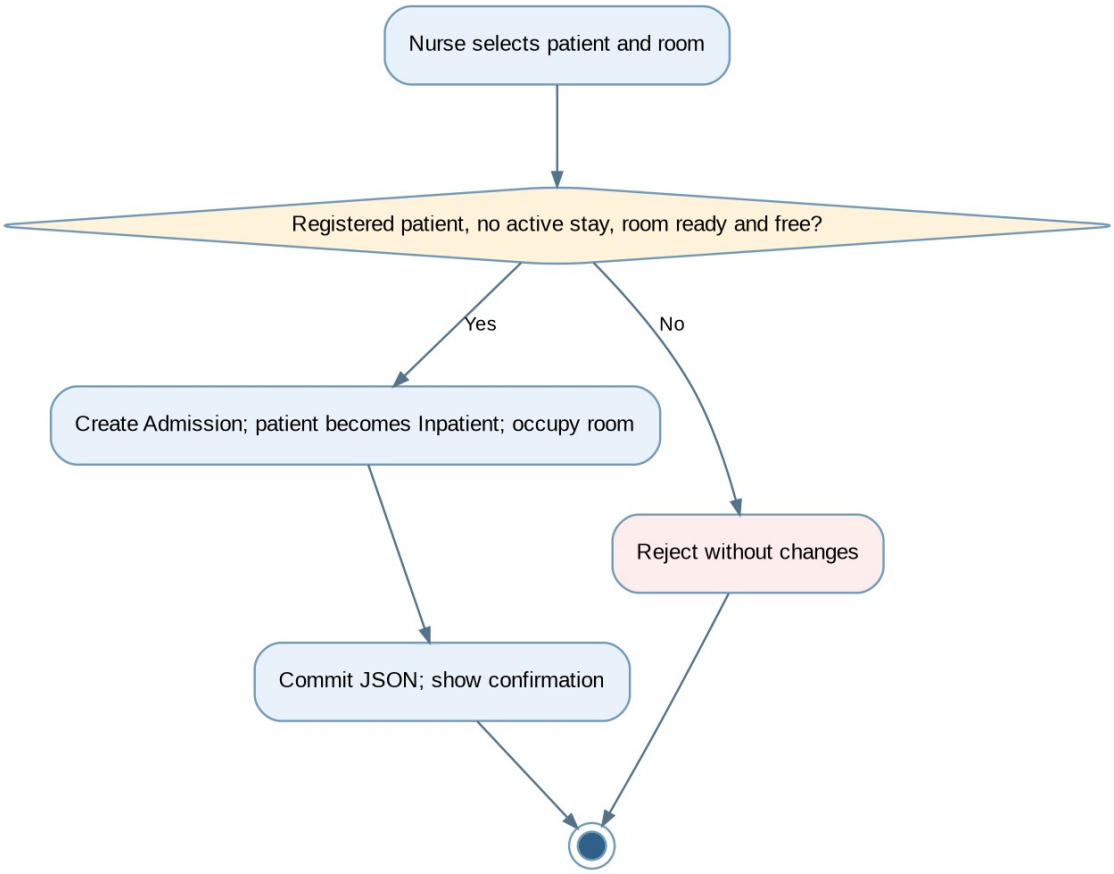


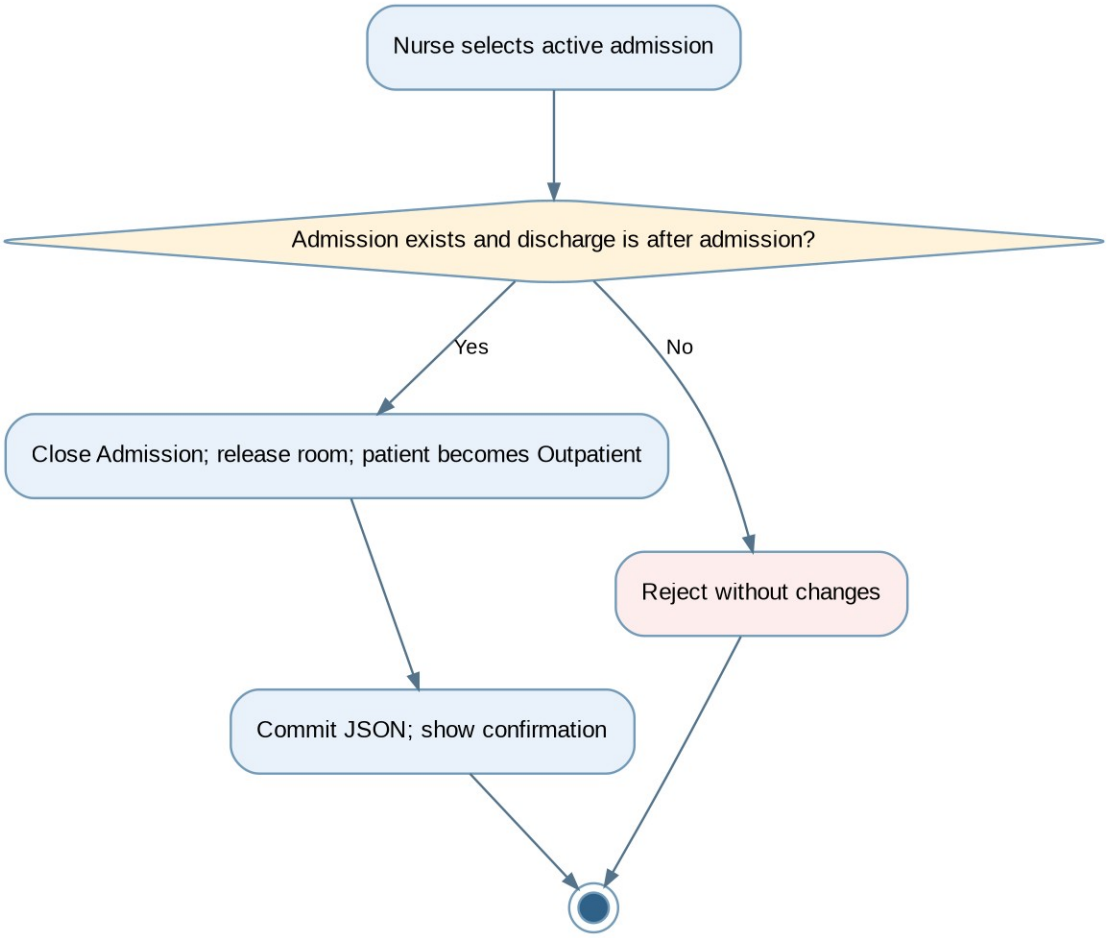

- Appointments are fixed one-hour slots. Doctor conflicts are checked before creation; a conflict returns alternate available slots and writes nothing.
- Recommended decision: only Scheduled records block new bookings; Cancelled records remain in history but release their slot. Completed records are historical and cannot be cancelled/rescheduled.
- Rescheduling validates the new slot first; on failure the original remains Scheduled. On success cancellation and replacement are saved as one logical transaction.

### 5.2 Medical history (UC4, UC5)

A Doctor selected by staff ID can record non-empty diagnosis text and prescriptions with medication, dosage and instructions. Records also retain doctor ID and timestamp; an appointment reference may be recorded when linked to a visit. Nurses can view medical histories but their menu cannot create diagnoses or prescriptions. All successful edits persist before the UI confirms.

### 5.3 Admission, discharge and room states (UC5)






Available means no active admission; Occupied means exactly one active admission. Admission validates the patient, absence of an active stay, and a free room. Discharge records the ending timestamp and makes the room available. All patient, admission and room changes are committed together. Proposed optional FR-26: maintenance is a separate flag; a vacant room in maintenance cannot accept admissions, and nurses cannot manually mark an occupied room as free.

### 5.4 Billing and payment (UC6)

Outpatient base charge = configured consultation fee. Inpatient base charge = daily room rate × billable days + nonnegative itemized treatment charges. Proposed billable days = max(1, ceil((discharge timestamp − admission timestamp) / 24 hours)); final inpatient billing occurs at discharge. These rounding and timing choices require instructor confirmation.

StandardPricing makes no discount. SeniorDiscountPricing applies a configured rate for an eligible age threshold; neither percentage nor threshold is established in earlier requirements. Persist the chosen rate, strategy, totals and payments. Use Decimal for currency. Accept payment only when 0 < amount ≤ outstanding balance; reject zero, negative and excessive payments without changing the balance.

## 6. JSON persistence and integrity

| Collection | Minimum fields |
|---|---|
| **patients** | patient_id, patient_type, name, dob, contact, medical_record_id, active_admission_id (nullable) |

| | |
|---|---|
| **staff / departments** | staff_id, staff_type, name, dob, contact, department_id, shift / department_id, name |
| **rooms / admissions** | room_id, daily_rate, maintenance_flag, occupied_by_patient_id; admission_id, patient_id, room_id, admitted_at, discharged_at |
| **appointments** | appointment_id, patient_id, doctor_id, start_time, status |
| **medical_records** | record_id, patient_id, diagnoses[] with doctor_id/time; prescriptions[] with medication/dosage/instructions/doctor_id/time |
| **bills** | bill_id, patient_id, source ID, base_amount, pricing_type, discount_rate, final_amount, payments[], balance |

Use one versioned top-level JSON snapshot so a booking, discharge or payment cannot partially persist across multiple files. Validate required IDs, references and subtype discriminators on load. Seed fictional reference data only for a genuinely new store; do not overwrite malformed existing data. Save to a temporary file and replace atomically when possible; preserve the last good snapshot on failure. Store ISO 8601 timestamps and Decimal values as strings.

## 7. Input validation and errors

| Condition | Required behavior |
|---|---|
| **Missing/invalid IDs, dates or mandatory fields** | Reject; show specific message; do not write |
| **Duplicate patient/staff/appointment IDs** | Reject without changing stored records |
| **Doctor conflict or invalid status transition** | Reject booking/change; preserve previous appointment |
| **Occupied/maintenance room or duplicate active stay** | Reject; preserve admission and room occupancy |
| **Missing diagnosis or prescription details** | Reject without modifying medical record |
| **Invalid rates, stay dates or payments** | Reject; preserve bill and balance |
| **Corrupt JSON or failed save** | Show error; do not silently reset store or claim success |

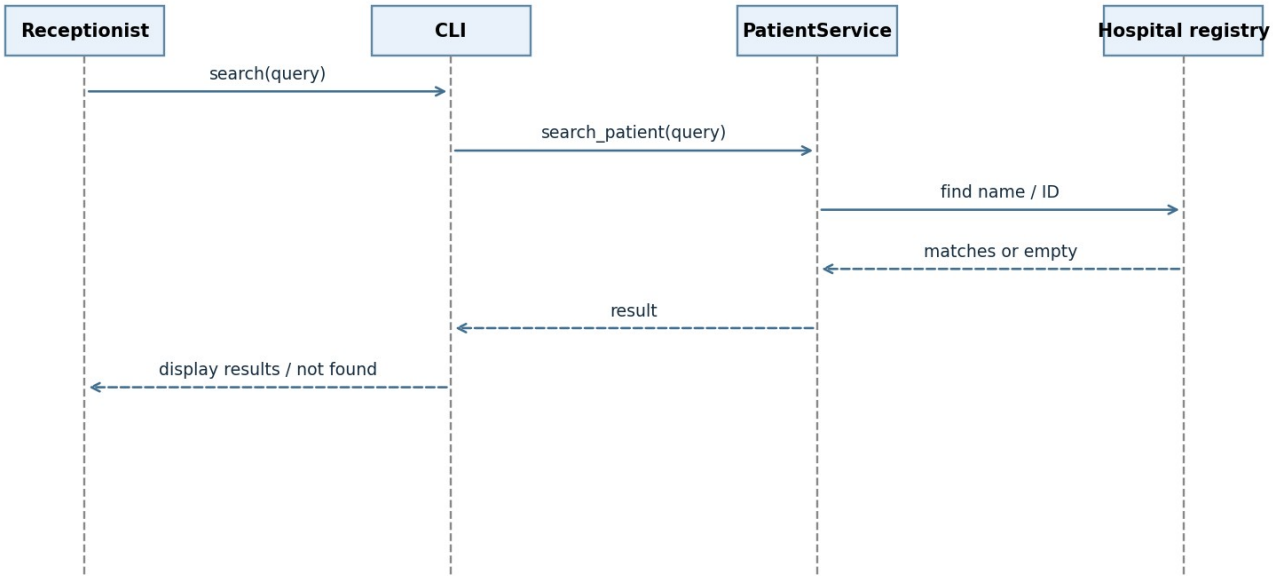
## 8. Console interface

Main menu: [1] Receptionist [2] Doctor [3] Nurse [4] Administrator [0] Exit. Doctor menu prompts for Doctor staff ID, then provides My Appointments, Medical History, Diagnosis, Prescription and Complete Appointment. Receptionist manages patients, bookings and billing; Nurse handles view history and admission/discharge; Administrator manages staff, departments and room records and reports. Each command validates input, delegates to a service, displays a result and returns to the role menu.

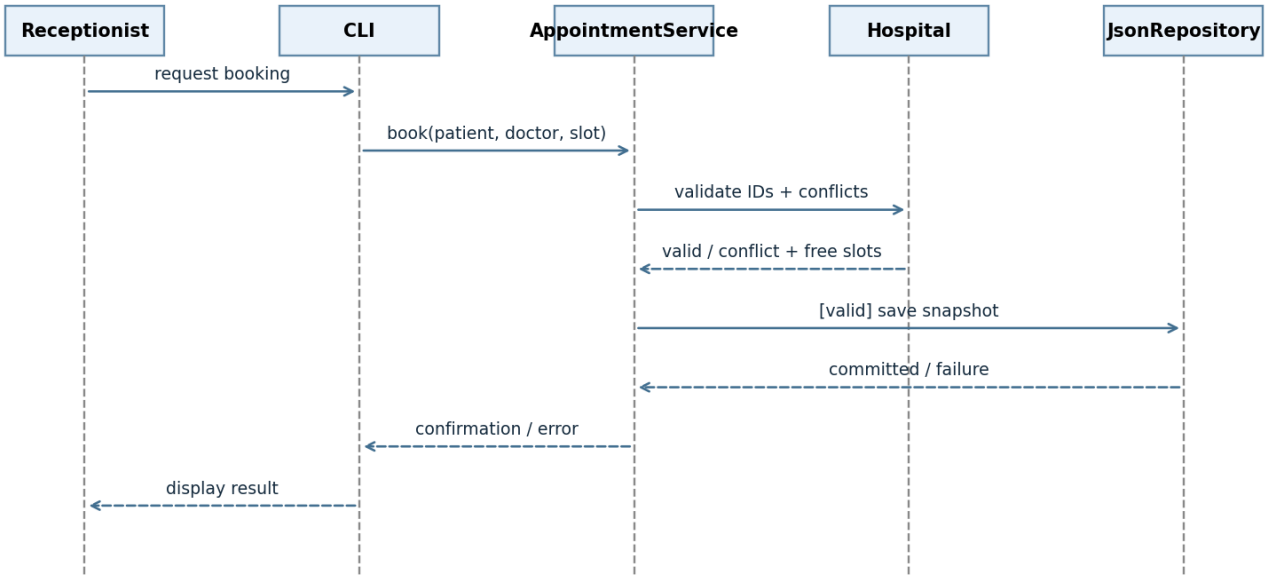
## 9. UML sequence diagrams

These diagrams show lifelines and chronological messages. Conditional saves occur only on successful validation; return arrows show results or errors.

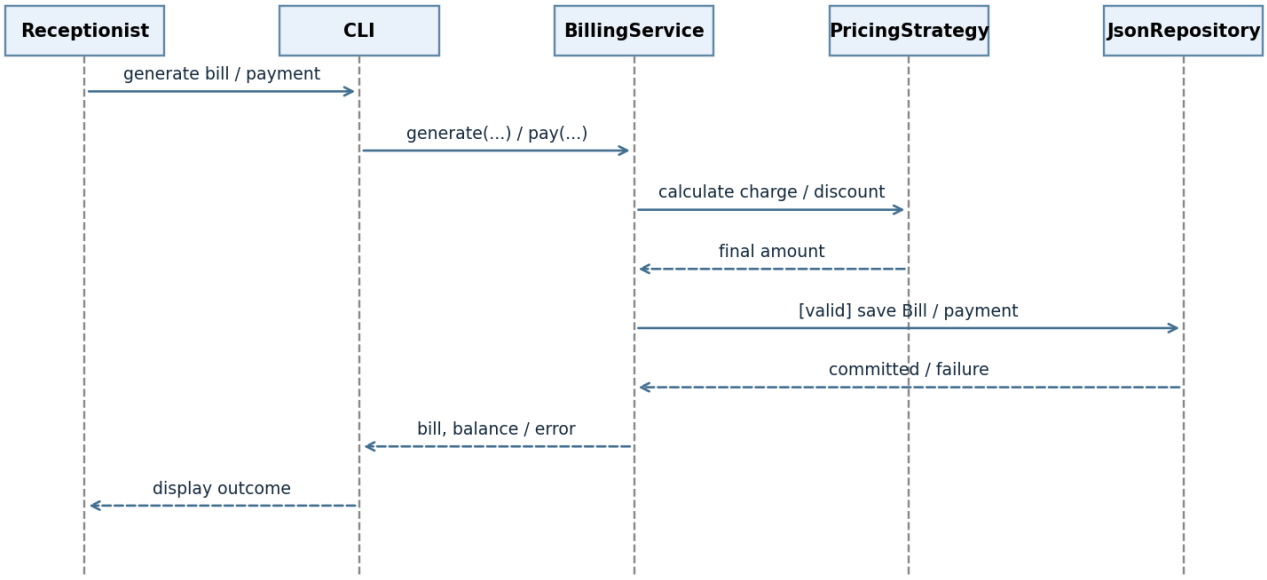
### Search patient (UC1)




### Book appointment (UC2)



### Generate bill / record payment (UC6)




## 10. Complete user-story and requirement traceability

| User story | Requirement | Use case | Design owner / proposed test |
|---|---|---|---|
| **US-PT-01** | FR-01 | UC1 | PatientService / TC-PT-01 |
| **US-PT-02** | FR-02 | UC1 | PatientService / TC-PT-02 |
| **US-PT-03** | FR-18 | UC1 | PatientService / TC-PT-03 |
| **US-AP-01** | FR-04 | UC2 | AppointmentService / TC-AP-01 |
| **US-AP-02** | FR-05 | UC2 | AppointmentService / TC-AP-02 |
| **US-AP-03** | FR-06 | UC3 | AppointmentService / TC-AP-03 |
| **US-AP-04** | FR-19 | UC3 | AppointmentService / TC-AP-04 |
| **US-AP-05** | FR-07 | UC4 | AppointmentService / TC-AP-05 |
| **US-AP-06** | FR-20 | UC4 | AppointmentService / TC-AP-06 |
| **US-MR-01** | FR-08 | UC4 | MedicalRecordService / TC-MR-01 |
| **US-MR-02** | FR-09 | UC4 | MedicalRecordService / TC-MR-02 |
| **US-MR-03** | FR-10 | UC4 | MedicalRecordService / TC-MR-03 |
| **US-MR-04** | FR-10 | UC5 | MedicalRecordService / TC-MR-04 |

| | | | |
|---|---|---|---|
| **US-AD-01** | FR-11 | UC5 | AdmissionService / TC-AD-01 |
| **US-AD-02** | FR-12 | UC5 | AdmissionService / TC-AD-02 |
| **US-AD-03** | FR-26 | UC5 | AdmissionService / TC-AD-03 |
| **US-BL-01** | FR-13 | UC6 | BillingService / TC-BL-01 |
| **US-BL-02** | FR-14 | UC6 | BillingService / TC-BL-02 |
| **US-BL-03** | FR-15 | UC6 | BillingService / TC-BL-03 |
| **US-BL-04** | FR-16 | UC6 | BillingService / TC-BL-04 |
| **US-ST-01** | FR-03 | UC7 | StaffFactory / TC-ST-01 |
| **US-ST-02** | FR-24 | UC7 | StaffService / TC-ST-02 |
| **US-ST-03** | FR-25 | UC7 | StaffService / TC-ST-03 |
| **US-RP-01** | FR-21 | UC8 | ReportService / TC-RP-01 |
| **US-RP-02** | FR-22 | UC8 | ReportService / TC-RP-02 |
| **US-RP-03** | FR-23 | UC8 | ReportService / TC-RP-03 |
| **US-RP-04** | FR-23 | UC8 | ReportService / TC-RP-04 |
| **US-DS-01** | FR-17 | All | JsonRepository / TC-DS-01 |

Every FR-01–FR-26 is represented; FR-10 and FR-23 map to two separate user stories each. Test identifiers denote planned pytest cases, not tests already executed.

## 11. Tests, constraints, and implementation order

| Test emphasis | Observable pass condition |
|---|---|
| **Doctor identity** | Non-doctor ID rejected; selected doctor's appointments only |
| **Booking and rescheduling** | Double booking rejected, alternatives shown, failed reschedule leaves original intact |
| **Clinical records** | Doctor changes persist; nurse cannot edit via nurse menu |
| **Admission and room** | One active patient per room; duplicate admission rejected; discharge frees room |
| **Pricing and payments** | Expected totals; zero/negative/over-balance payments rejected |
| **Persistence** | Restart recreates subclasses, links, diagnoses, admissions and balances |

Implementation order: Person/Patient/Staff and core models → JSON repository → patient/staff services → appointment service → medical record service → admissions → pricing/billing → reports → CLI → pytest and UML/code reconciliation. Each Must Have acceptance criterion requires a pytest case before the project is declared implemented.

## 12. Design decisions to confirm and completion checklist

- Confirm patient subtype conversion at admission/discharge (a concrete Patient plus Admission is an alternative).
- Confirm cancelled-slot reuse and whether historical Completed appointments ever constrain booking.
- Confirm senior eligibility age/discount percentage and which charges are discounted.

- Confirm inpatient day rounding, treatment-charge entry and when inpatient bills finalize.
- Confirm optional room maintenance policy and allowed nurse status actions.

The current proposal makes these choices explicit so implementation can be designed and tested; they are not represented as previously approved instructor rules.

Submission design is complete when: all approved stories map to design elements; UML diagrams reflect intended code; component responsibilities are separated; workflow failures do not mutate persistent data; JSON structures and error messages are specified; and important uncertainties are recorded rather than hidden. After implementation, rerun pytest and update UML to actual class/method names.

Source documents: HPMS Project Proposal (30 September 2026); HPMS Requirements Collection (7 October 2026); HPMS User Stories (17 October 2026); instructor SP3 feedback (89/100).